FININTELL AI — SMART FINANCIAL SAAS

Full Software Engineering Lifecycle, Agile Roadmap, HLD/LLD & Database Architecture

ENGINEERING SPECIFICATION OVERVIEW
Target Project: FinIntell AI (Financial Intelligence & Automated Invoice Parsing SaaS)
Technology Stack: Java 21 (Spring Boot 3), React 19 (TypeScript), PostgreSQL (pgvector), Spring AI / LLM APIs
Methodology: Agile Scrum Model (4 Sprints / 4-Week Production Lifecycle)
Author / Role: Lead Full-Stack Software Engineer & System Architect

1. PROBLEM STATEMENT & SYSTEM REQUIREMENTS

Business Context: Freelancers, small business owners, and growing agency founders spend over 10–15 hours every month manually extracting transaction details from receipt PDFs, categorizing expenses, reconciling vendor invoices, and tracking cash flow trends. Existing accounting tools are either overly complex enterprise systems or simple spreadsheets lacking automated data entry and natural language insight.

Proposed Solution: *FinIntell AI* is an automated financial management SaaS platform. It combines a high-throughput Java 21 Spring Boot microservices backend with a responsive React 19 frontend and an integrated AI document processing pipeline. Users can upload receipt images/PDFs, automatically extract structured line items, manage budgets, and query financial analytics in natural language.

Module	Functional Requirement	Technical Implementation
Authentication & Security	Stateless User Registration, Login & Role-Based Access Control (RBAC)	Spring Security 6, JWT Token Guard Filters, BCrypt Password Hashing
Financial Core	Multi-currency Expense & Income CRUD, Budget Alerting, Vendor Analytics	Spring Data JPA, PostgreSQL Relational Schema, ACID Transactions
AI Receipt Parser	Upload PDF/Images, auto-extract vendor, date, total, items to JSON	Spring AI, OpenAI/Claude API Structured Outputs, Multipart Storage
RAG Budget Assistant	Conversational Q&A over historical financial records in natural language	PostgreSQL pgvector, Vector Embeddings, LangChain/Spring AI RAG Pipe
Analytics Dashboard	Interactive visual charts for spending velocity, budget limits, & forecasting	React 19, TypeScript, Tailwind CSS, Recharts / Chart.js

2. AGILE SDLC MODEL & 4-SPRINT IMPLEMENTATION PLAN

The project follows the **Agile Scrum methodology** with 1-week Sprints, daily standups, backlog grooming, and explicit Definition of Done (DoD) criteria. This approach ensures continuous iteration, risk mitigation, and early feature validation.

Sprint / Phase	Sprint Goal & Key Focus	Deliverables & Acceptance Criteria
Sprint 1 (W1) **Backend Foundation**	Database Modeling, Persistence Layer & REST CRUD APIs	PostgreSQL Schema, Spring Boot 3 App setup, User/Transaction CRUD APIs verified with Postman.

Sprint / Phase	Sprint Goal & Key Focus	Deliverables & Acceptance Criteria
Sprint 2 (W2) **Auth & Frontend**	JWT Security, React SPA, State Management & UI Integration	Spring Security JWT Filter, React + TypeScript Dashboard, Form Validation, Axios Interceptors.
Sprint 3 (W3) **AI Engine & RAG**	Receipt Parsing Automation & Conversational Analytics RAG	Multipart File Endpoint, AI JSON Parsing Service, pgvector Embeddings, Interactive Chat UI.
Sprint 4 (W4) **DevOps & Release**	Unit/Integration Testing, Dockerization & Cloud Deployment	JUnit5/Mockito Test Suite, Docker Compose setup, Live Vercel Frontend & Render Backend.

Definition of Done (DoD) Standard:

- All backend code covered by >80% unit/integration tests.
- REST APIs documented via OpenAPI / Swagger UI specs.
- Zero high-severity security vulnerabilities or unhandled exceptions.
- Continuous Integration (CI) build passes cleanly on GitHub Actions.

3. HIGH-LEVEL DESIGN (HLD) ARCHITECTURE

The system employs a **3-Tier Microservice-Ready Architecture** built for security, horizontal scalability, and strict separation of concerns.

System Layer	Component / Technology	Architectural Responsibilities
1. Client Tier	React 19, TypeScript, Tailwind CSS, Axios	Single Page Application (SPA), state management, client-side validation, UI components, chart renderings.
2. API Gateway & Security	Spring Security, CORS Guard, JWT Filter	Authentication verification, token extraction, request rate limiting, CORS headers, security context propagation.
3. Application Tier	Spring Boot 3 Services, DTO Mappers	Business logic execution, transaction processing, input validation, exception handling, data mappers.
4. AI & LLM Service	Spring AI, OpenAI/Claude API, pgvector	Asynchronous PDF OCR receipt parsing, structured JSON formatting, RAG vector search, LLM orchestration.
5. Data Persistence	PostgreSQL 16, Spring Data JPA, HikariCP	Relational transaction records, user credentials, category mappings, indexing, database connection pooling.

End-to-End Execution Sequence for AI Receipt Parsing:

- **Step 1 (Client Upload):** User drops a receipt PDF/Image into the React UI. Axios posts `multipart/form-data` to `/api/v1/ai/parse-receipt` with Bearer JWT.
- **Step 2 (Security Validation):** `JwtAuthenticationFilter` validates token signature and attaches `UserPrincipal` to Security Context.
- **Step 3 (AI Service Request):** `ReceiptParserService` buffers the file, encodes image bytes, and calls LLM API with structured output prompt JSON schema.
- **Step 4 (Database Transaction):** Extracted JSON payload (Vendor, Date, Amount, Items) is saved transactionally into PostgreSQL `transactions` and `receipt_items` tables.
- **Step 5 (Client Sync):** Spring Boot returns `201 Created` with `TransactionDTO`. React state updates optimistically, refreshing charts immediately.

4. LOW-LEVEL DESIGN (LLD) & CLASS ARCHITECTURE

Low-Level Design establishes explicit controller endpoints, service interfaces, Data Transfer Objects (DTOs), and centralized exception handling.

HTTP	Endpoint Path	Request Body / Params	Response	Description
POST	/api/v1/auth/register	RegisterRequestDTO	201 Created	Registers new user with BCrypt hashed password.

HTTP	Endpoint Path	Request Body / Params	Response	Description
POST	/api/v1/auth/login	AuthRequestDTO	200 OK + JWT	Authenticates user and returns JWT bearer token.
GET	/api/v1/transactions	page, size, category, startDate	200 OK (Page)	Fetches paginated user transactions with filters.
POST	/api/v1/transactions	TransactionCreateDTO	201 Created	Creates manual income/expense entry.
POST	/api/v1/ai/parse-receipt	MultipartFile file	201 Created	AI OCR receipt parsing & auto-creation.
POST	/api/v1/ai/chat	ChatQueryDTO	200 OK (Stream)	Conversational financial query over pgvector RAG.

Core Java Class & Interface Hierarchy:

```
// Global Exception Handling Structure
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse("NOT_FOUND", ex.getMessage(), LocalDateTime.now()));
    }
    @ExceptionHandler(AiParsingException.class)
    public ResponseEntity<ErrorResponse> handleAiError(AiParsingException ex) {
        return ResponseEntity.status(HttpStatus.UNPROCESSABLE_ENTITY)
            .body(new ErrorResponse("AI_PARSE_FAILED", "Failed to extract receipt data", LocalDateTime
.now()));
    }
}
```

5. DATABASE DESIGN (RELATIONAL SCHEMA & SQL)

The relational storage engine utilizes **PostgreSQL 16** with the `pgvector` extension. The schema enforces foreign key integrity, cascade options, and indexing for fast spatial time-series lookups.

DATABASE ENTITY STRUCTURE
Entity Relationships (ERD Summary):
• **USERS** (1) ■■■< (N) **TRANSACTIONS** (Foreign Key: `user_id`)
• **CATEGORIES** (1) ■■■< (N) **TRANSACTIONS** (Foreign Key: `category_id`)
• **TRANSACTIONS** (1) ■■■< (N) **RECEIPT_ITEMS** (Foreign Key: `transaction_id`)
• **USERS** (1) ■■■< (N) **FINANCIAL_EMBEDDINGS** (pgvector for RAG search)

Production DDL SQL Schemas & Performance Indexing:

```
-- 1. Users Table
CREATE TABLE users (
    id BIGSERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(100) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2. Categories Table
CREATE TABLE categories (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    type VARCHAR(20) CHECK (type IN ('EXPENSE', 'INCOME')),
    icon_key VARCHAR(50)
);

-- 3. Transactions Table (Core Ledger)
CREATE TABLE transactions (
    id BIGSERIAL PRIMARY KEY,
    user_id BIGINT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    category_id INT REFERENCES categories(id),
    amount NUMERIC(12, 2) NOT NULL,
    transaction_date DATE NOT NULL,
    merchant_name VARCHAR(150),
    description TEXT,
    receipt_url VARCHAR(500),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 4. Indexing for High-Performance Queries
CREATE INDEX idx_trans_user_date ON transactions(user_id, transaction_date DESC);
CREATE INDEX idx_trans_merchant ON transactions(user_id, merchant_name);
```

6. TESTING STRATEGY, DEVOPS & DEPLOYMENT

A complete software engineering lifecycle demands rigorous testing standards and automated continuous deployment pipelines.

Domain	Tooling / Framework	Implementation Strategy
Unit Testing	JUnit 5, Mockito, AssertJ	Mocks service layer dependencies, tests business logic rules, math edge cases, and exception pathways.
Integration Testing	Testcontainers, MockMvc	Spins up real PostgreSQL container instances during testing to validate JPA queries & SQL joins.
Containerization	Docker, Docker Compose	Multi-stage Dockerfile builds lightweight Spring Boot JAR image (<250MB) and multi-container Compose setup.
CI/CD Pipeline	GitHub Actions Workflow	Automates linting, test execution, Docker build, and deployment triggers upon git push to `main`.
Cloud Hosting	Vercel & Render / Railway	Vercel hosts static React 19 SPA with global CDN; Render/Railway hosts Spring Boot app & Managed Postgres.

NEXT STEPS FOR PORTFOLIO EXECUTION

Summary Action Checklist for Freelance Developers:

1. Clone backend repository template and configure PostgreSQL instance via Docker Compose.
2. Implement Phase 1 Data Models and verify REST endpoints with Postman test suites.
3. Wire Spring Security JWT authentication and build React 19 dashboard UI.
4. Integrate Spring AI receipt parsing API and deploy live demo link to Vercel/Render for client showcase.